GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering & Technology

Skip List Optimization

Search and update operations, probabilistic vs. deterministic analysis, and randomized data structures

Student Name	_____	Enrollment No.	_____
Subject	Advanced Data Structure (METCE11502)	Semester	1 (M.Tech)
Activity	ALA 1	Marks	10

1. Aim

To explore the **search and update operations on skip lists**, to analyse the **probabilistic and deterministic aspects** of skip lists, and to study different methods of **randomizing data structures and algorithms** for optimization. All algorithms were implemented in Python and the measurements in this report were obtained by running that code.

2. Introduction

A **skip list** (W. Pugh, 1990) is an ordered dictionary built from a sorted linked list plus several *express lanes*. Level 1 contains every key; level 2 contains a subset that acts as shortcuts over level 1; level 3 is a subset of level 2, and so on. A search starts on the highest level, moves right while the next key is smaller than the target, drops one level when it cannot move further, and finally reaches the target on level 1.

The height of each node is chosen **at random**: a new node is promoted to the next level with probability *p* (normally 1/2), like repeatedly tossing a coin. This randomisation gives balanced-tree performance (expected O(log n)) without any rotations or rebalancing code.

Skip list with 10 keys (p = 0.5); red path = search(19)

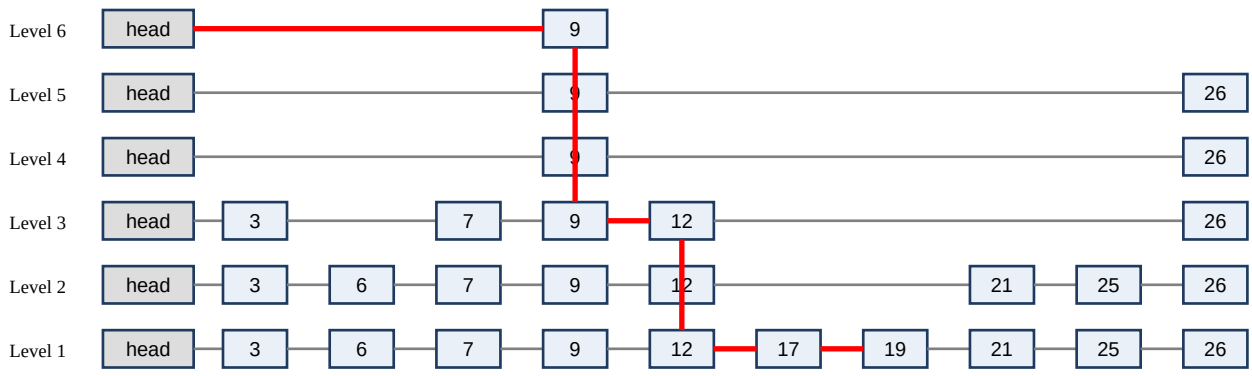


Figure 1: Skip list built by the program in Section 5 (node heights come from the actual run). The search for 19 uses the high levels to skip over most keys.

3. Operations and Complexity

Operation	Algorithm (summary)	Expected time
Search(k)	Start at head on the top level. While next key < k move right; otherwise drop one level. At level 1, check whether the next node holds k.	O(log n)
Insert(k)	Search for k while remembering the last node visited on every level (*update[ ]*). Generate a random height h. Splice the new node into levels 1..h using update[ ].	O(log n)
Update(k, v)	Search for k; if the node exists, overwrite its value. (Insert with an existing key also acts as an update.)	O(log n)
Delete(k)	Search with update[ ]; unlink the node on every level where it appears; shrink empty top levels.	O(log n)
Space	Each node has on average 1/(1–p) forward pointers (2 for p = 1/2).	O(n)
Worst case	All operations can degrade to O(n) in theory, but only with extremely small probability.	O(n) (very unlikely)

4. Implementation (Python)

The complete source code is given below (`skiplist.py`). It contains the probabilistic skip list with search, insert, update and delete, a counter of key comparisons used for the analysis, a builder for the deterministic (ideal) skip list used in Section 7, and a sorted linked list used as a baseline.

```
import random


class SkipNode:
    __slots__ = ("key", "value", "forward")

    def __init__(self, key, value, level):
        self.key = key
        self.value = value
        self.forward = [None] * level          # forward[i] = next node at level i


class SkipList:
    """Probabilistic skip list: each new node gets height h with probability p^(h-1)(1-p)."""

    def __init__(self, p=0.5, max_level=24, seed=None):
        self.p = p
        self.max_level = max_level
        self.level = 1                          # current highest level in use
        self.head = SkipNode(None, None, max_level)
        self.size = 0
        self.rng = random.Random(seed)
        self.steps = 0                          # key comparisons (for analysis)

    def random_level(self):
        lvl = 1
        while self.rng.random() < self.p and lvl < self.max_level:
            lvl += 1
        return lvl

    def search(self, key):
        x = self.head
        for i in range(self.level - 1, -1, -1):        # top level -> bottom level
            while x.forward[i] is not None and x.forward[i].key < key:
                x = x.forward[i]
                self.steps += 1
            self.steps += 1
        x = x.forward[0]
        if x is not None and x.key == key:
            return x.value
        return None

    def insert(self, key, value=None, level=None):
```

```
        update = [self.head] * self.max_level           # last node visited on each level
        x = self.head
        for i in range(self.level - 1, -1, -1):
            while x.forward[i] is not None and x.forward[i].key < key:
                x = x.forward[i]
            update[i] = x
        nxt = x.forward[0]
        if nxt is not None and nxt.key == key:          # key exists -> UPDATE its value
            nxt.value = value
            return False
        lvl = level if level is not None else self.random_level()
        if lvl > self.level:
            self.level = lvl                            # new levels start from head
        node = SkipNode(key, value, lvl)
        for i in range(lvl):                            # splice node into each level
            node.forward[i] = update[i].forward[i]
            update[i].forward[i] = node
        self.size += 1
        return True

    def update(self, key, value):
        """Update the value of an existing key; returns False if key is absent."""
        x = self.head
        for i in range(self.level - 1, -1, -1):
            while x.forward[i] is not None and x.forward[i].key < key:
                x = x.forward[i]
        x = x.forward[0]
        if x is not None and x.key == key:
            x.value = value
            return True
        return False

    def delete(self, key):
        update = [None] * self.max_level
        x = self.head
        for i in range(self.level - 1, -1, -1):
            while x.forward[i] is not None and x.forward[i].key < key:
                x = x.forward[i]
            update[i] = x
        x = x.forward[0]
        if x is None or x.key != key:
            return False
        for i in range(self.level):
            if update[i].forward[i] is not x:
                break
            update[i].forward[i] = x.forward[i]         # unlink on every level
        while self.level > 1 and self.head.forward[self.level - 1] is None:
            self.level -= 1                              # shrink empty top levels
        self.size -= 1
        return True

    def keys(self):
        out, x = [], self.head.forward[0]
        while x:
            out.append(x.key)
            x = x.forward[0]
        return out

    def level_counts(self):
        """Number of nodes present on each level (level 1 = all nodes)."""
        counts = []
        for i in range(self.level):
            c, x = 0, self.head.forward[i]
            while x:
                c += 1
                x = x.forward[i]
            counts.append(c)
        return counts

    def show(self, max_items=20):
        for i in range(self.level - 1, -1, -1):
            row, x = [], self.head.forward[i]
            while x and len(row) < max_items:
                row.append(str(x.key))
                x = x.forward[i]
            print(f"Level {i + 1}: head -> " + " -> ".join(row) + " -> None")
```

```
def ideal_skip_list(keys, p=0.5):
    """DETERMINISTIC (perfectly balanced) skip list built from sorted keys:
    the i-th key (1-based) gets height = 1 + (number of trailing zero bits of i)."""
    sl = SkipList(p=p)
    for i, k in enumerate(sorted(keys), start=1):
        h = 1
        while i % (2 ** h) == 0 and h < sl.max_level:
            h += 1
        sl.insert(k, k, level=h)
    return sl


class SortedLinkedList:
    """Baseline for comparison: search = linear scan."""

    def __init__(self, keys):
        self.keys = sorted(keys)
        self.steps = 0

    def search(self, key):
        for k in self.keys:
            self.steps += 1
            if k >= key:
                return k == key
        return False
```

5. Demonstration of Search and Update Operations

Output produced by running the operations on a skip list with 10 keys (random seed 7):

```
After inserting 10 keys:
Level 6: head -> 9 -> None
Level 5: head -> 9 -> 26 -> None
Level 4: head -> 9 -> 26 -> None
Level 3: head -> 3 -> 7 -> 9 -> 12 -> 26 -> None
Level 2: head -> 3 -> 6 -> 7 -> 9 -> 12 -> 21 -> 25 -> 26 -> None
Level 1: head -> 3 -> 6 -> 7 -> 9 -> 12 -> 17 -> 19 -> 21 -> 25 -> 26 -> None
search(19)      -> v19
search(20)      -> None
update(19)      -> search(19) = NEW-19
insert(17,'dup')-> returns False (existing key: value updated, no new node)
delete(7)       -> True | keys: [3, 6, 9, 12, 17, 19, 21, 25, 26]
delete(100)     -> False
Nodes per level : [9, 7, 4, 2, 2, 1]
```

Observations: the topmost level contains very few nodes, searching for an absent key (20) returns None, update changes only the stored value, inserting an existing key does not create a second node, and delete unlinks the node from every level where it was present.

6. Probabilistic Analysis

6.1 Theory

- A node reaches level *h* with probability p^{h-1}. The expected number of nodes on level h is $n \cdot p^{h-1}$, so the list has about $\log_{1/p} n$ levels ($\log_2 n$ for p = 1/2).
- Expected pointers per node = 1/(1–p). For p = 1/2 this is 2, i.e. about the same memory as a binary tree.
- Expected search cost ≈ $(\log_{1/p} n)/p$ comparisons: climbing back up from the bottom takes about 1/p steps per level (a geometric random variable). For p = 1/2 this is about $2 \cdot \log_2 n$.
- The cost depends only on the random coin tosses, **not on the order or distribution of the keys**. No input can be chosen by an adversary to force bad behaviour (the adversary cannot see the coin tosses).

6.2 Experiment A – search cost versus n

Random keys were inserted in random order (p = 0.5) and the average number of key comparisons per search was measured over 800 successful searches. The deterministic (ideal) skip list of Section 7 is shown for comparison.

n	Levels used	Random skip list (measured)	Ideal skip list (measured)	Theory $(\log_2 n)/p$	$\log_2 n$
1000	11	19.4	15.0	19.9	10.0
4000	11	23.5	18.0	23.9	12.0
16000	14	28.7	21.0	27.9	14.0
64000	17	31.2	23.9	31.9	16.0

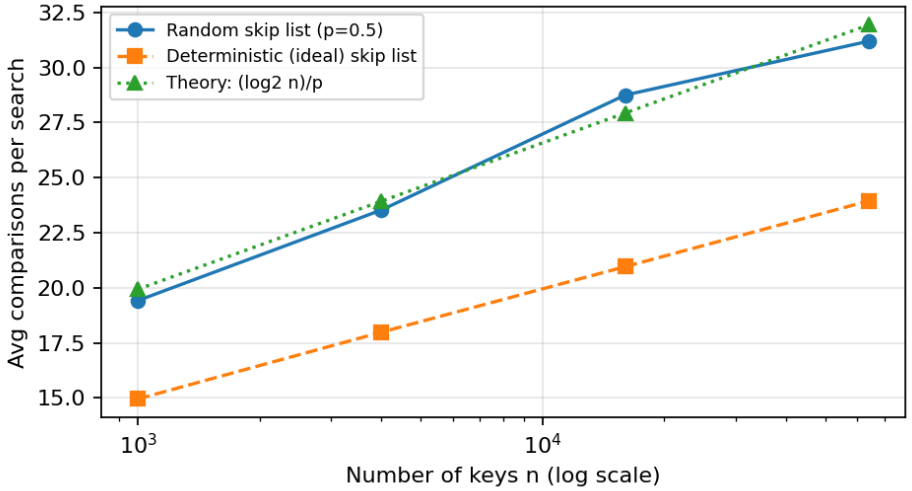


Figure 2: The measured cost of the random skip list follows the theoretical curve and grows logarithmically.

For comparison, an ordinary sorted linked list needed on average **2180.0** comparisons per search for n = 4000, compared with about 24 for the skip list.

6.3 Experiment B – effect of the promotion probability p

n = 16000 keys. A smaller p means fewer, taller-spaced express lanes (less memory, more steps per level); a larger p means more pointers but a shorter walk per level.

p	Levels used	Avg search comparisons (measured)	Theory $(\log_{1/p} n)/p$	Avg pointers/node (measured)	Theory 1/(1−p)
0.25	7	25.1	27.9	1.34	1.33
0.5	17	29.6	27.9	2.01	2.0
0.75	24	47.6	44.9	4.02	4.0

Search cost is smallest near p = 1/2 (or 1/e), and memory use matches 1/(1–p). For p = 0.75 the program's level cap (24) is lower than the theoretical number of levels, which slightly distorts that row. p = 1/2 is therefore the best general-purpose choice; p = 1/4 is attractive when memory matters.

6.4 Experiment C – variability of a randomized structure

The same experiment (n = 4000) was repeated with 40 different random seeds:

Measure	Minimum	Mean	Maximum	Std. deviation
Avg comparisons per search	21.9	25.0	34.1	2.55

Measure	Minimum	Mean	Maximum	Std. deviation
Number of levels	11	14.1	24	-

Performance is good on average but not identical on every run: a few unlucky random sequences give a taller list (here the program's cap of 24 levels was reached in one run) and a larger cost. The spread is small compared with the O(n) cost of a linked list, which is why skip lists are described as having *O(log n) with high probability*.

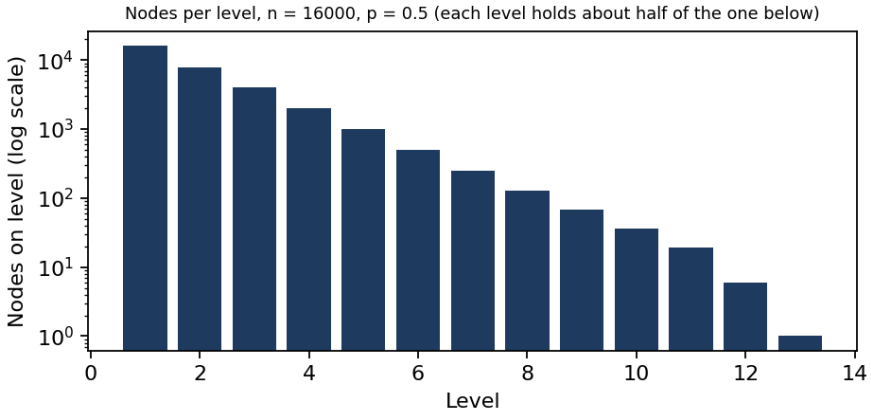


Figure 3: Each level contains about half the nodes of the level below, as predicted by the geometric distribution. (Level counts for 16000 keys inserted in sorted order.)

7. Deterministic Aspects

A skip list does not have to be random. In a **deterministic (ideal) skip list** the height of the i-th key is fixed: *height = 1 + (number of trailing zero bits of i)*. Level 2 then holds every 2nd key, level 3 every 4th key, and so on – the perfect 'binary-search' layout. It was built from sorted keys by the function `ideal_skip_list()` in Section 4.

Aspect	Probabilistic skip list	Deterministic (ideal) skip list
Node heights	Random (coin tosses)	Computed from the key's rank
Search cost	Expected ≈ $2 \cdot \log_2 n$; varies between runs	Guaranteed ≤ about $2 \cdot \log_2 n$; measured about $1.5 \cdot \log_2 n$ (Table in 6.2)
Insert / Delete	Local change only: O(log n) expected	Inserting one new smallest key changed the height of **1000 of 1000** nodes: O(n) re-balancing
Dependence on input order	None	Needs sorted/known rank information
Implementation	Very simple	Complex when updates must be supported

The experiment shows the trade-off: determinism gives guaranteed and slightly cheaper searches, but keeping the perfect layout after an update is expensive. **Deterministic skip lists** studied in the literature (Munro, Papadakis and Sedgewick, 1992) relax the perfect layout (for example, allowing 1–3 nodes between express-lane nodes) so that updates only touch O(log n) nodes while the worst case stays O(log n). The probabilistic version obtains a similar effect with far less code, which is its main attraction.

8. Methods of Randomizing Data Structures and Algorithms

Randomization is used to make the *expected* performance independent of the input, so that no particular input is a bad case. Three experiments illustrate the idea.

8.1 Randomized choice of levels (skip list) versus an unbalanced search tree

2000 keys were inserted in two orders into (i) a plain binary search tree (no balancing) and (ii) a skip list.

Insertion order	Plain BST height	Skip list avg comparisons per search	Skip list levels
Sorted order	1999	27.1	16
Random order	24	25.7	16

Sorted input turns the BST into a chain of height 1999, but the skip list is unaffected because its structure depends on coin tosses, not on the keys.

8.2 Randomized pivot selection in Quicksort

Quicksort was run on n = 1500 integers and the number of element comparisons was counted ($n \cdot \log_2 n \approx 15826$).

Input	Pivot rule	Comparisons
Already sorted	First element	1,125,749
Already sorted	Random element	18,040
Random order	First element	18,496
Random order	Random element	17,701

With the first element as pivot, an already sorted input needs about $n^2/2$ comparisons. A randomly chosen pivot makes the cost about $n \cdot \log n$ on *every* input, including the sorted one.

8.3 Summary of randomization techniques

Technique	Where the randomness is used	Benefit
Random levels (skip list)	Height of each new node	Expected O(log n) search/insert/delete without rotations
Random pivot (Quicksort / Quickselect)	Choice of the partitioning element	Expected O(n log n) on all inputs; no bad input
Randomized BST / Treap	Random priority per key, heap-ordered	Tree is balanced like a random BST
Universal hashing	Hash function chosen at random from a family	Expected O(1) per operation for any key set
Randomized algorithms (Las Vegas / Monte Carlo)	Random choices inside the algorithm	Simplicity and speed; correct always (Las Vegas) or with high probability (Monte Carlo)

Need for randomization: deterministic structures need extra balancing rules (rotations, rebuilding) to resist worst-case inputs. Randomization achieves the same expected bounds with simpler code and removes the dependence on the input; the price is that guarantees are probabilistic and results can vary from run to run.

9. Conclusion

- Search, insert, update and delete on a skip list all run in O(log n) expected time; the implementation is much shorter than an AVL or red-black tree.
- Measured search cost matched the theory: about $(\log_2 n)/p$ comparisons, with p = 1/2 giving the best balance of time and memory.

- The randomized structure is insensitive to the order of the input (BST height 1999 on sorted input, skip list unaffected) but individual runs vary slightly.
- A deterministic skip list gives guaranteed and cheaper searches but is expensive to update (all 1000 nodes changed height after one insertion).
- Randomization (random levels, random pivots, random priorities, universal hashing) is a general optimisation technique that makes expected performance independent of the input.

References

- W. Pugh, “Skip lists: a probabilistic alternative to balanced trees”, *Communications of the ACM*, 33(6), 1990.
- J. I. Munro, T. Papadakis, R. Sedgewick, “Deterministic skip lists”, *Proc. ACM-SIAM SODA*, 1992.
- M. T. Goodrich, R. Tamassia, *Algorithm Design*, John Wiley, 2002 (skip lists and randomized algorithms).
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press.
- M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 2nd ed., Pearson, 2004.